

Spark Session - Configuração de Workers e Memória

Exemplo Básico

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MeuJob") \
    .config("spark.executor.instances", "8") \
    .config("spark.executor.cores", "4") \
    .config("spark.executor.memory", "8g") \
    .config("spark.driver.memory", "4g") \
    .config("spark.driver.cores", "2") \
    .getOrCreate()
```

Configurações Principais

Config	Descrição	Exemplo
spark.executor.instances	Quantidade de workers	"8"
spark.executor.cores	CPUs por worker	"4"
spark.executor.memory	RAM por worker	"8g"
spark.driver.memory	RAM do driver (coordenador)	"4g"
spark.driver.cores	CPUs do driver	"2"

Configurações Adicionais Úteis

Config	Descrição	Exemplo
<code>spark.executor.memoryOverhead</code>	Memória extra (off-heap)	"1g"
<code>spark.sql.shuffle.partitions</code>	Partições no shuffle	"200"
<code>spark.default.parallelism</code>	Paralelismo padrão	"100"
<code>spark.dynamicAllocation.enabled</code>	Alocação dinâmica	"true"
<code>spark.dynamicAllocation.minExecutors</code>	Mínimo de executors	"2"
<code>spark.dynamicAllocation.maxExecutors</code>	Máximo de executors	"20"

Exemplos por Cenário

Job Pequeno (desenvolvimento)

```
spark = SparkSession.builder \  
  .appName("Dev") \  
  .config("spark.executor.instances", "2") \  
  .config("spark.executor.cores", "2") \  
  .config("spark.executor.memory", "4g") \  
  .config("spark.driver.memory", "2g") \  
  .getOrCreate()
```

Job Médio (Silver/Gold)

```
spark = SparkSession.builder \  
  .appName("SilverGold") \  
  .config("spark.executor.instances", "8") \  
  .config("spark.executor.cores", "4") \  
  .config("spark.executor.memory", "16g") \  
  .config("spark.driver.memory", "8g") \  
  .getOrCreate()
```

Job Pesado (ABT Builder com 95M registros)

```
spark = SparkSession.builder \  
  .appName("ABTBuilder") \  
  .config("spark.executor.instances", "16") \  
  .config("spark.executor.cores", "4") \  
  .config("spark.executor.memory", "32g") \  
  .config("spark.driver.memory", "16g") \  
  .config("spark.sql.shuffle.partitions", "400") \  
  .getOrCreate()
```

Alocação Dinâmica

Para clusters que variam a carga:

```
spark = SparkSession.builder \  
  .appName("Dinamico") \  
  .config("spark.dynamicAllocation.enabled", "true") \  
  .config("spark.dynamicAllocation.minExecutors", "2") \  
  .config("spark.dynamicAllocation.maxExecutors", "20") \  
  .config("spark.dynamicAllocation.initialExecutors", "4") \  
  .getOrCreate()
```

Ambientes Gerenciados

Ambiente	Observação
Databricks	Configs definidas no cluster, não na session
OCI Data Flow	Definido no job (driver_shape, executor_shape, num_executors)
EMR	Configs no cluster ou --conf no spark-submit
Local	spark.master = "local[*]" usa todos os cores

Exemplo Databricks (via cluster)

```
# No Databricks, a session já vem configurada pelo cluster
spark = SparkSession.builder.getOrCreate()

# Para verificar configs atuais:
spark.sparkContext.getConf().getAll()
```

Exemplo OCI Data Flow

```
# Definido no OCIDataFlowOperator (Airflow) ou console OCI
OCIDataFlowOperator(
    driver_shape="VM.Standard.E4.Flex",
    executor_shape="VM.Standard.E4.Flex",
    num_executors=8,
    driver_shape_config={"ocpus": 4, "memory_in_gbs": 32},
    executor_shape_config={"ocpus": 4, "memory_in_gbs": 32},
)
```

Cálculo de Recursos

Regra geral:

```
Memória Total = executor.instances × executor.memory
Cores Totais = executor.instances × executor.cores
```

Exemplo:

- 8 executors × 16GB = 128GB total
- 8 executors × 4 cores = 32 cores total